TP3 · GlobalLogistics

Despliegue y rendimiento de Gestión de Envíos

Trabajo Práctico 3 · Arquitectura de Sistemas · Universidad Champagnat

Integrantes: Lozano, Siccardi y Rivamar

Repositorio público de entrega: https://github.com/LozanoSh/TP3-GlobalLogistics

Objetivo

Desplegar con Docker Compose un balanceador Nginx y tres réplicas del mismo microservicio REST; comprobar identidad, salud y distribución de solicitudes; medir latencia, throughput y fallos bajo 100 usuarios y bajo saturación de CPU.

Arquitectura e infraestructura como código

Un Nginx publica el puerto 8080 y distribuye en round-robin a api1, api2 y api3. Las tres instancias comparten la imagen tp3-globallogistics-api:local y sólo son accesibles por la red interna de Compose. Los cuatro servicios tienen health checks. Las réplicas son fijas; no hay autoescalado.

Cliente → Nginx :8080 → api1 :3000 | api2 :3000 | api3 :3000

1 balanceador · 3 réplicas · 1 imagen de API · 4 contenedores

Evidencia de despliegue

```
PS C:\Users\LENOVO\OneDrive\Documentos\ChatGPT\arquitectura de sistemas\TP3-GlobalLogistics> docker compose ps
NAME                          IMAGE                              COMMAND                  SERVICE   CREATED         STATUS
              PORTS
tp3-globallogistics-api1-1    tp3-globallogistics-api:local   "docker-entrypoint.s…"   api1      3 minutes ago   Up 3 mi
nutes (healthy)   3000/tcp
tp3-globallogistics-api2-1    tp3-globallogistics-api:local   "docker-entrypoint.s…"   api2      3 minutes ago   Up 3 mi
nutes (healthy)   3000/tcp
tp3-globallogistics-api3-1    tp3-globallogistics-api:local   "docker-entrypoint.s…"   api3      3 minutes ago   Up 3 mi
nutes (healthy)   3000/tcp
tp3-globallogistics-nginx-1   nginx:1.27-alpine               "/docker-entrypoint.…"   nginx     3 minutes ago   Up 2 mi
nutes (healthy)   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp
PS C:\Users\LENOVO\OneDrive\Documentos\ChatGPT\arquitectura de sistemas\TP3-GlobalLogistics>
```

Captura original de la ejecución de Rivamar: cuatro contenedores saludables. La validación actual repitió el arranque y la prueba funcional; los IDs de contenedor actuales difieren de los de la captura.

Microservicio, balanceo y salud

POST /envios/validar comprueba en memoria el contrato de una solicitud de envío internacional y responde 200 o 400. No crea ni persiste el envío. GET /api/instancia devuelve hostname e ID real de contenedor; GET /health informa la salud de la API y GET /api/cpu?work=N ejecuta cálculo síncrono. Nginx ofrece además /nginx-health.

Tres instancias observadas

Capturas originales de respuestas individuales atravesando el balanceador:

GET http://localhost:8080/api/instancia — carga 1 de 3 — 2026-10-29T22:32:11.277Z


```
{"instancia":"api1","contenedor_id":"f4d456f905de5f81d9b6894d2fed66701b52c60a8e93501bd95528f550f23296","instance_id":"api1","pi
d":1,"servicio":"tp3-globallogistics-api","version":"1.0.0","node":"v22.23.4","uptime_s":372,"timestamp":"2026-10-29T22:32:11.255
Z"}
```


Respuesta de api1 con identidad del contenedor.

GET http://localhost:8080/api/instancia — carga 2 de 3 — 2026-10-29T22:32:11.332Z


```
{"instancia":"api2","contenedor_id":"a1767ac1239d82dcc8d374ca3f78aafd2ff9cfbf94291c83fe27fdacccaaf3d7","instance_id":"api2","pi
d":1,"servicio":"tp3-globallogistics-api","version":"1.0.0","node":"v22.23.4","uptime_s":372,"timestamp":"2026-10-29T22:32:11.321
Z"}
```


Respuesta de api2 con identidad del contenedor.

GET http://localhost:8080/api/instancia — carga 3 de 3 — 2026-10-29T22:32:11.380Z


```
{"instancia":"api3","contenedor_id":"8d669a60a335281f460aa3d0254707d92f4a59e6cf52c109395e180f81fa2608","instance_id":"api3","pi
d":1,"servicio":"tp3-globallogistics-api","version":"1.0.0","node":"v22.23.4","uptime_s":372,"timestamp":"2026-10-29T22:32:11.370
Z"}
```


Respuesta de api3 con identidad del contenedor.

Validación funcional

El script node tests/validacion_envios.js comprobó nueve POST válidos distribuidos entre las tres réplicas, cinco entradas inválidas con HTTP 400, y respuestas HTTP 200 de /health, /api/instancia y /api/cpu?work=1. Resultado actual: VALIDACION ENVIOS OK. Las 28 pruebas unitarias pasaron.

Metodología de rendimiento

k6 generó una referencia de 5 usuarios virtuales (VU) durante 30 s, validación moderada con 100 VU sostenidos 45 s y carga de CPU con work=100 a 1, 10 y 25 VU durante 30 s cada nivel. Todo el tráfico pasó por Nginx. Se registraron puntos crudos de k6, resúmenes, docker stats, probes de salud e identidad. Las fuentes originales permanecen en tests/results/.

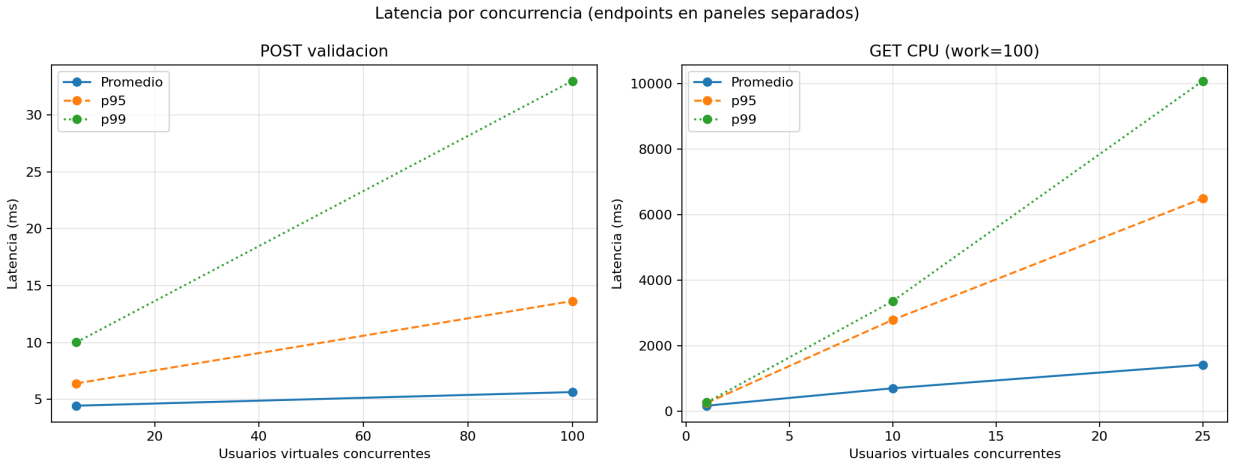
Resultados medidos

Las series de validación y CPU corresponden a operaciones distintas. Los valores siguientes provienen de tests/results/resumen_escenarios.csv y los resúmenes k6 de cada corrida. Latencias en milisegundos.

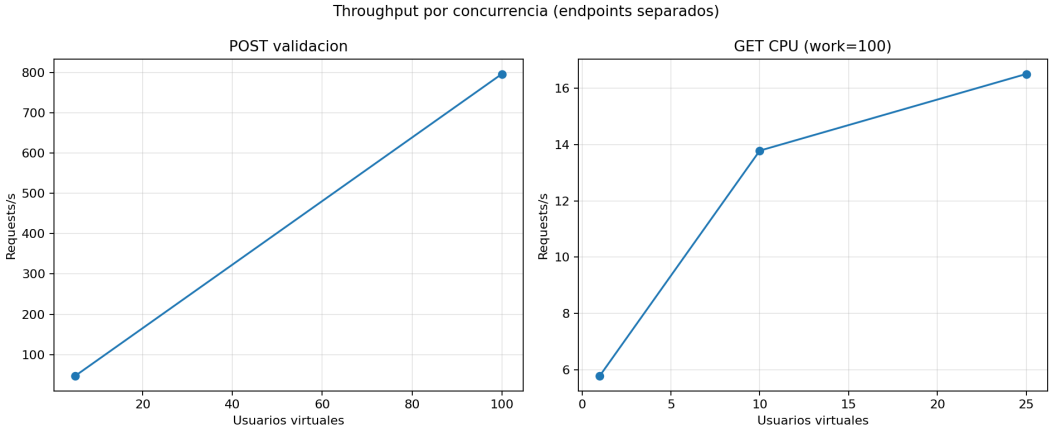
Escenario	VU	Requests	req/s	Prom.	p95	p99	Máx.	Fallos
Validación ref.	5	1.425	47,34	4,45	6,40	10,01	33,36	0
Validación	100	51.807	796,02	5,64	13,64	32,98	131,09	0
CPU work=100	1	174	5,79	171,98	255,00	273,42	288,42	0
CPU work=100	10	438	13,78	704,30	2.792,49	3.363,93	6.470,15	0
CPU work=100	25	553	16,51	1.421,45	6.499,47	10.084,81	11.621,87	0

Los 100 VU de validación dieron 100 % de checks correctos. En CPU, k6 tampoco registró solicitudes fallidas, aunque la salud se deterioró a 25 VU.

Latencia y capacidad



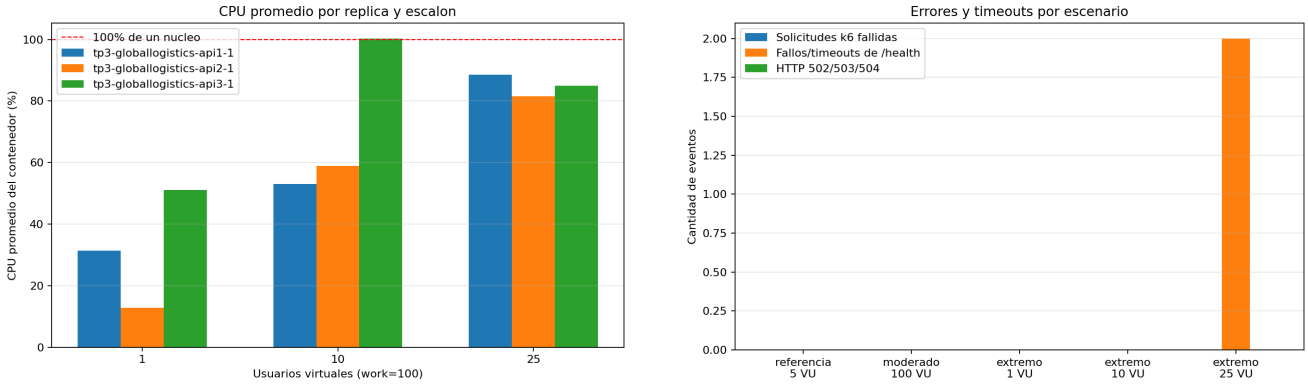
Promedio, p95 y p99 por concurrencia; cada endpoint se grafica en su propio panel.



Throughput: a 25 VU la serie CPU alcanzó 16,51 req/s, con un costo fuerte de latencia.

Saturación, salud y restricciones temporales

A 10 VU de CPU, p95 creció de 255 a 2.792 ms frente a 1 VU y una réplica alcanzó 104,31 % de CPU muestreada. A 25 VU, los picos de las tres réplicas rondaron o superaron 100 %; p95 alcanzó 6,50 s, p99 10,08 s y la máxima 11,62 s.



CPU promedio por réplica (izquierda) y fallos de carga/probes (derecha). Los gráficos se generaron con los CSV y puntos k6 originales.

Probes de salud bajo CPU

VU	Muestras /health	No 200 / timeout	> 1 s	Demora máxima
1	30	0	0	75,44 ms
10	21	0	5	2.709,64 ms
25	15	2	5	4.034,41 ms

Análisis y conclusión

La degradación clara comienza a 10 VU para /api/cpu?work=100. El cálculo síncrono ocupa el hilo principal de cada proceso Node.js; las solicitudes posteriores, incluidos health checks, esperan en su event loop. Las tres réplicas fijas distribuyen trabajo, pero todas compiten por CPU. A 25 VU hubo dos timeouts de salud; la prueba se detuvo allí sin extrapolar a niveles mayores.

La máxima de 11.621,87 ms supera desde 10 VU la referencia temporal de 1.000 ms usada en el TP2. La validación con 100 VU llegó a 131,09 ms de máxima, pero /envios/validar no implementa la creación persistente del TP2; estas mediciones no demuestran su SLA. Nginx sigue siendo un punto único de entrada y no hay autoescalado. La arquitectura cumple el despliegue y balanceo pedidos, con la limitación observada bajo CPU intensiva.